

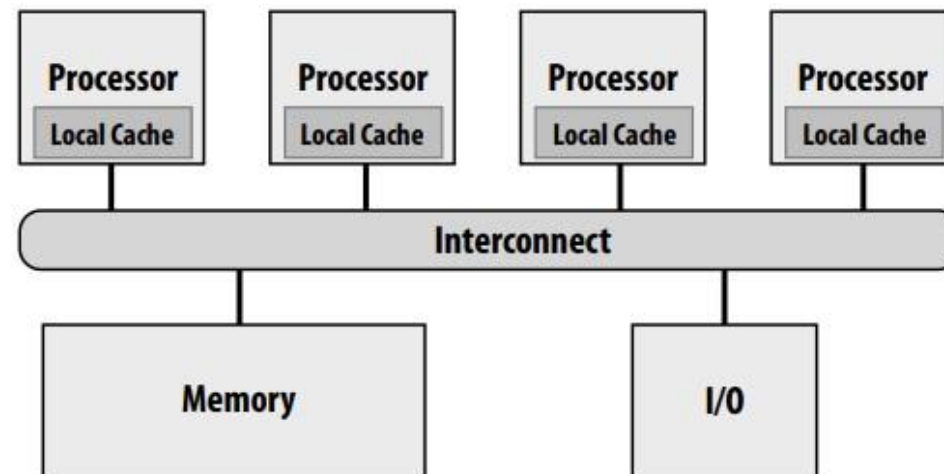
Cache coherence

Bibliography

- Culler et. al. – Chapters 5.1-5.4;8.1-8.3

Shared memory multiprocessor

- Symmetric Multiprocessor or SMP
- Hardware directly supports the shared address space programming model
- Message passing programming model supported by an intervening software layer
- Multiprogramming support in OS



Shared memory multiprocessor

- Processors read and write to shared variables
- Expected: a read from address X will return the last value that was written at address X by any processor
- Processors replicate the contents of memory in cache
- It is possible that processors will observe a different value for the same memory location

Example

Proc						Cache P ₁	Cache P ₂	Cache P ₃	Main M
P ₁	Read u					U=5			U=5
P ₃		Read u				U=5		U=5	U=5
P ₃			Write u=7			U=5		U=7	U=7
P ₁				Read u		U=5			U=7
P ₂					Read u		U=7		U=7

- 3 processors SMP
- Assume write-through policy
- Question: What happens in the case of write-back policy?

Memory coherence problem

- A read from address X will return the last value that was written at address X by any processor
- Problem: the memory system consists of a shared main memory and replicated private caches -> abstraction of a single shared memory space

Single CPU coherence problem

- DMA data transfers
- CPU and I/O use shared buffers
- Problem: data written in buffer is in cache and is not updated in the main memory -> I/O reads old data
- Solution: do not cache pages that contain shared buffers (OS solution)

Coherence-definition

- A memory system is coherent if the results of any execution of a program are such that, for each location, it is possible to construct a hypothetical serial order of all operations to the location (i.e., put all reads/writes issued by all processors into a total order) which is consistent with the results of the execution and in which
 - operations issued by any particular processor occur in the above sequence in the order in which they were issued to the memory system by that processor, and
 - the value returned by each read operation is the value written by the last write to that location in the above sequence.

Coherence

- Write propagation: notification of a write has to reach all processors
- Write serialization: all writes to a location are seen in the same order by all processors.
- Important: the order in which things appear to happen, as detectable from the results of an execution

Shared caches

- Eliminates the problem of replication
- Problem: scalability
- Benefits:
 - One processor may pre-fetch cache lines for other processors
 - Facilitates fine-grained sharing

Coherence vs consistency

MEMORY COHERENCE

- Ensures consistency within a single memory location
- Ensures that all copies of the same memory location are updated properly across different caches.
- Ensures all processors see the latest value of a memory location.

MEMORY CONSISTENCY

- Defines the ordering of memory operations across the system
- Defines the expected ordering of memory operations across multiple threads or processors.
- Prevents data races and inconsistencies in parallel computing.

Memory consistency

<u>P1</u>	<u>P2</u>
/* Assume initial value of A and flag is 0 */	
A = 1;	while (flag == 0); /* spin idly */
flag = 1;	print A;

- Which will be the result of the execution?
- What change will P2 see first? A or flag?
 - Update order of A and flag NOT implied by coherence
 - Need for an ordering model...

Memory consistency model

- A memory consistency model specifies constraints on the order in which memory operations must appear to be performed
- Sequential Consistency – strictest model
 - All operations appear in a single global order
 - Simple but can reduce performance
- Relaxed Consistency – optimized for performance
 - Allow reordering for efficiency
 - Require synchronization (e.g., barriers)

Sequential Consistency

- The result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor occur in this sequence in the order specified by its program.
- Memory appears to service the requests of memory access one at a time in an interleaved manner according to an arbitrary schedule
- Operations are atomic: the results appear to all processors after completion

Sufficient conditions for preserving sequential consistency

1. Every process issues memory requests in the order specified by the program
2. After a write operation is issued, the issuing process waits for the write to complete before issuing its next operation
3. After a read operation is issued, the issuing process waits for the read to complete, and for the write whose value is being returned by the read to complete, before issuing its next operation.

Cache coherence through bus snooping

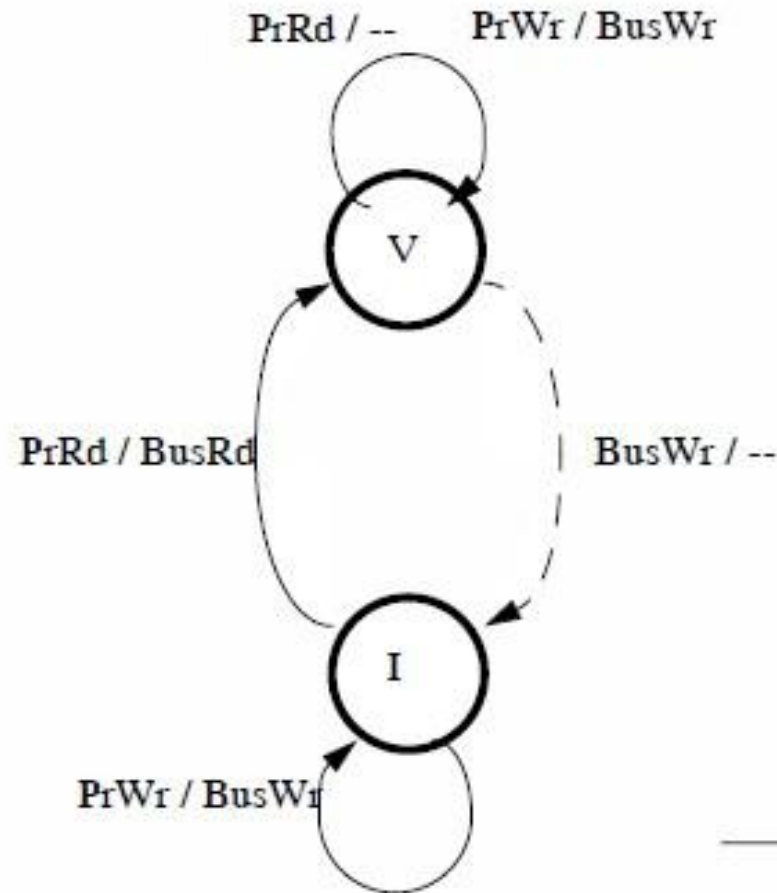
- Cache controllers 'snoop' on the bus and monitor the transactions
- Cache controller takes action if a bus transaction is relevant
 - Invalidate cache line
 - Update value in own cache
- Snooping-based cache coherence scheme ensures that:
 - all "necessary" transactions appear on the bus
 - each cache monitors the bus for relevant transactions and takes suitable actions.

Simple snoop-based implementation

- Assume write-through
- Cache line state: V-valid; I-invalid
- Cache controller invalidates the cache line if it detects a write transaction

Proc						Cache P ₁	Cache P ₂	Cache P ₃	Main M
P ₁	Read u					u=5(V)			u=5
P ₃		Read u				u=5(V)		u=5(V)	
P ₃			Write u=7			u=5(I)		u=7(V)	u=7
P ₁				Read u		u=7			
P ₂					Read u		u=7		

Write-through invalidation - state diagram



- States of cache line:

- Valid (V)
- Invalid (I)

- Processor requests:

- reads (PrRd)
- writes (PrWr)

- Bus transactions:

- BusRd: The cache controller puts the address on the bus and asks for a copy
- BusWr: The cache controller puts the address and the contents for the memory block on the bus

—▶ Processor initiated transactions
--▶ Bus-snooper initiated transactions

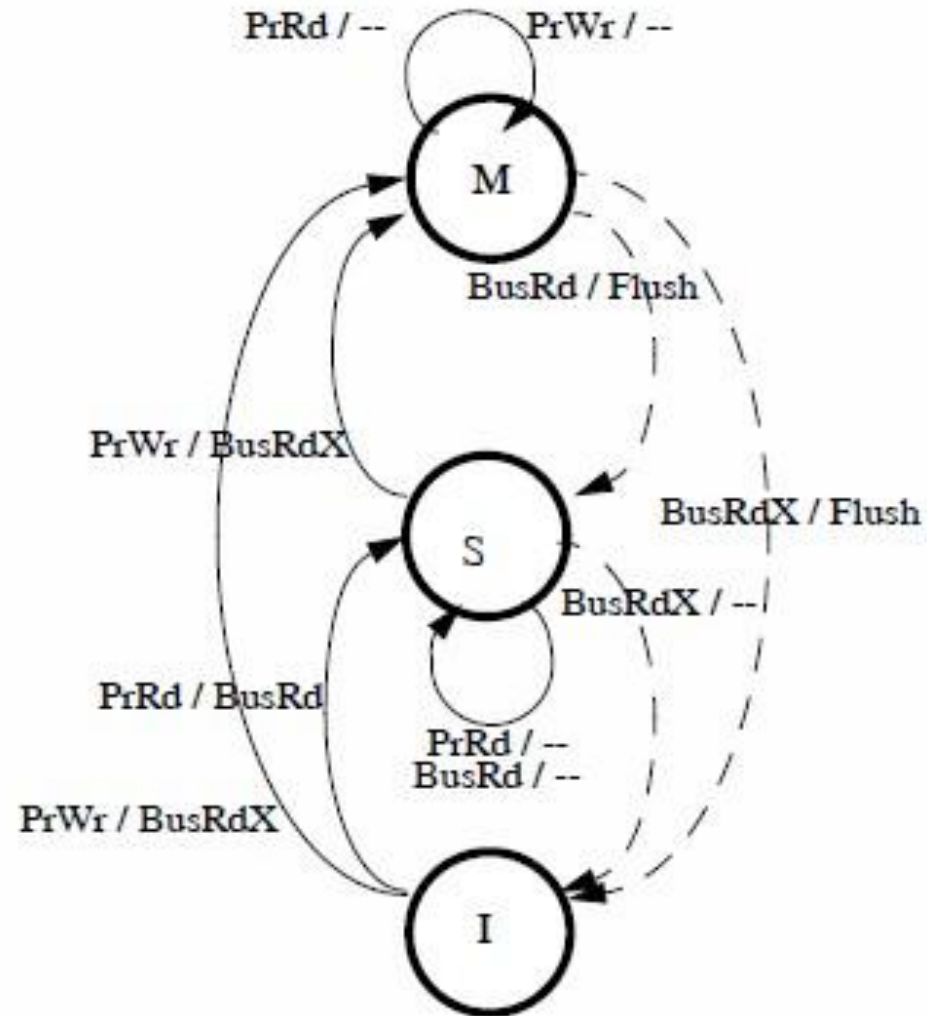
Write-through invalidation

- Assumptions
 - Writes visible to all cache controllers in the same order
 - Memory transactions are atomic
 - Invalidation applied immediately after receiving broadcast
- Write-through: inefficient
- Write-back: how to insure write propagation?

MSI- write-back invalidation protocol

- States of cache line:
 - Modified (M) – only this processor has a valid copy
 - Shared (S) - zero or more other caches may also have an up-to-date (shared) copy
 - Invalid (I)
- Processor requests:
 - reads (PrRd)
 - writes (PrWr)
- Bus transactions:
 - Bus Read (BusRd): The cache controller puts the address on the bus and asks for a copy that it does not intend to modify
 - Bus Read-Exclusive (BusRdX): The cache controller puts the address on the bus and asks for an exclusive copy that it intends to modify
 - Writeback (BusWB): The cache controller puts the address and the contents for the memory block on the bus

MSI-state diagram



MSI protocol

- Write propagation:
 - When in M state, flush line if any read from other processor
 - Invalidate if read-exclusive
- Write serialization
 - Writes in the order of BusRdX
 - Reads in order of BusRd
 - Other writes only in M state

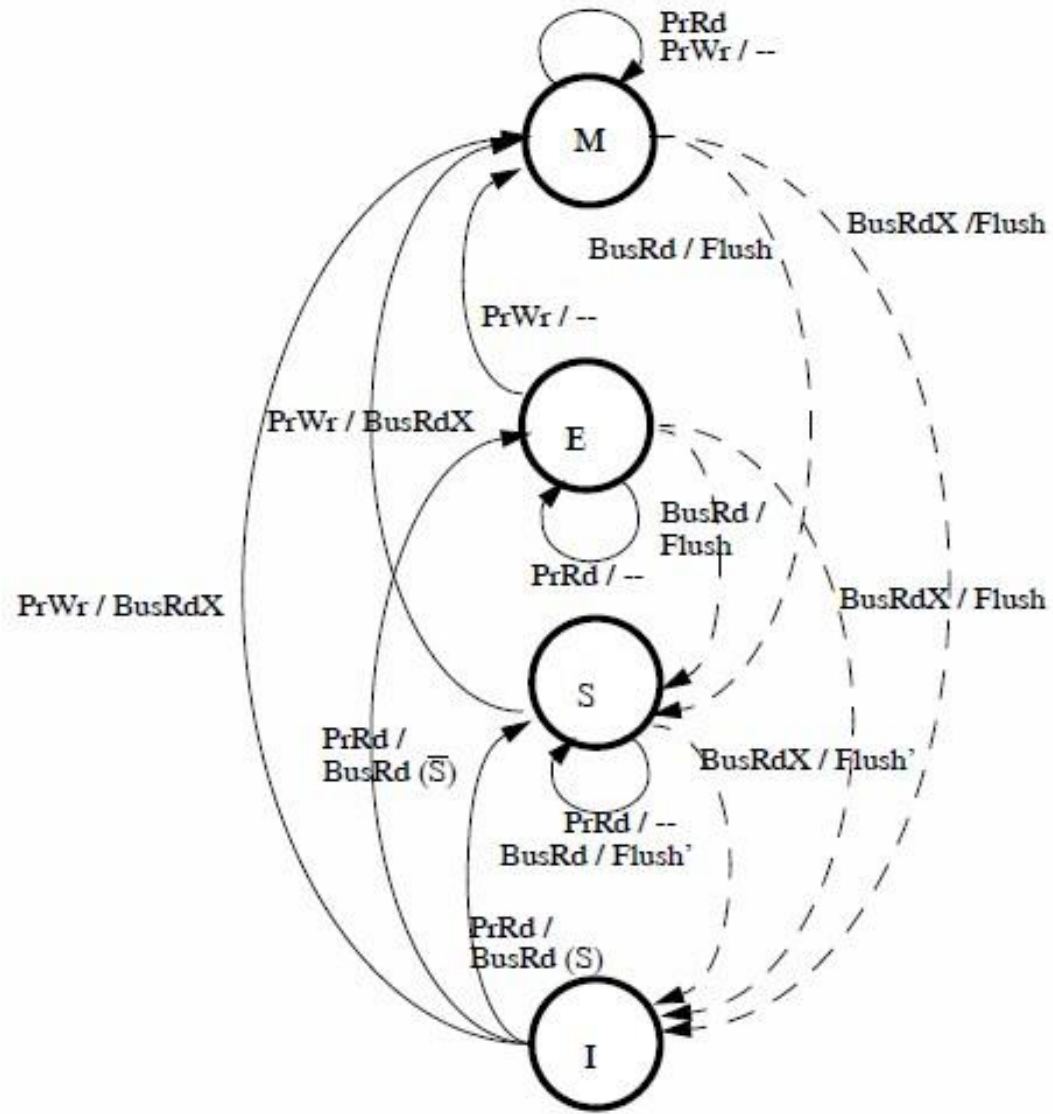
Example

<u>Proc. Action</u>	<u>State in P1</u>	<u>State in P2</u>	<u>State in P3</u>	<u>Bus Action</u>	<u>Data Supplied By</u>
1. P1 reads u	S	--	--	BusRd	Memory
2. P3 reads u	S	--	S	BusRd	Memory
3. P3 writes u	I	--	M	BusRdX	Memory
4. P1 reads u	S	--	S	BusRd	P3's cache
5. P2 reads u	S	S	S	BusRd	Memory

MESI- write-back invalidation protocol

- MSI inefficiency: process reads and then modifies data
 - Two transactions: BusRd (I->S), BusRdX(S->M)
 - Even if data is not shared
- Save the second transaction by introducing new state: exclusive (no other processor is caching the block)
- States:
 - Modified (M)
 - Exclusive-clean (E)
 - Shared (S)
 - Invalid (I)
- The same bus transactions

MESI – state diagram

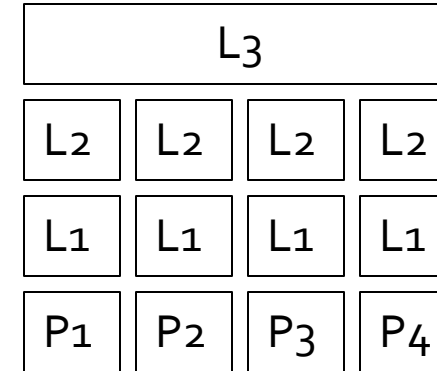


Lower-level choices

- Who should supply the block for a BusRd transaction when both the memory and another cache have a copy of it?
 - **Cache-to-cache sharing**
- Higher complexity of implementation, but faster (reduces latency and required memory bandwidth)

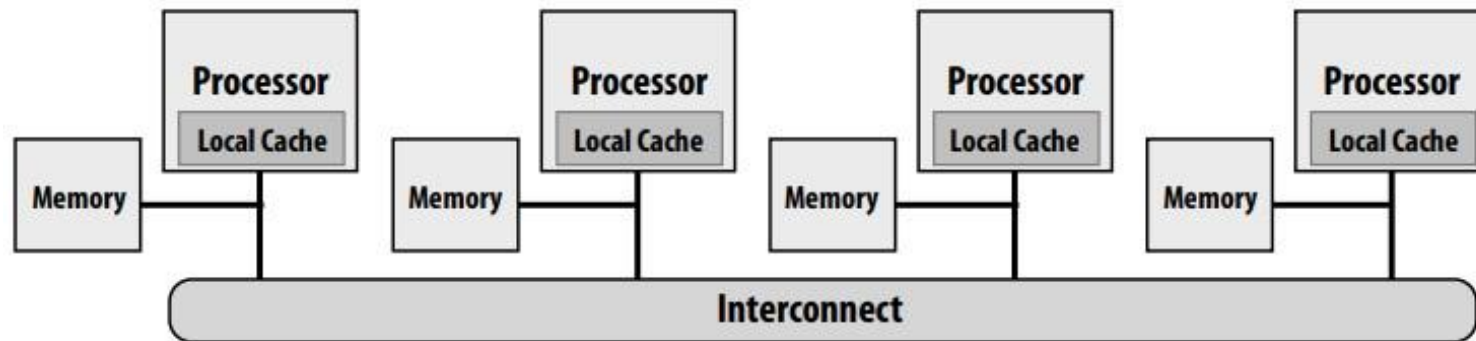
Real case

- Multi-level cache hierarchies
- Changes made in L1 may not be visible for cache controller in L2
- Inefficient to snoop buses independently
- **Scalability of snoop-based protocols is limited by the ability to broadcast coherence messages to all caches**



Large machines

- Locating regions of memory near processors increases scalability (NUMA)
- cc-NUMA: cache-coherent non-uniform memory access

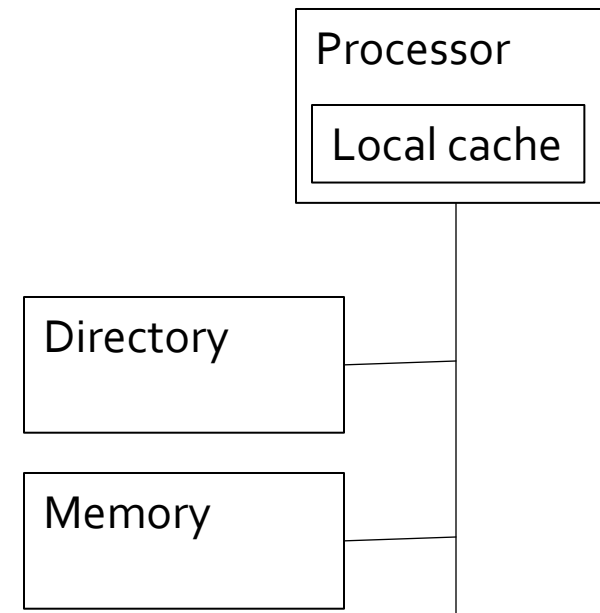


Directory-based cache coherence

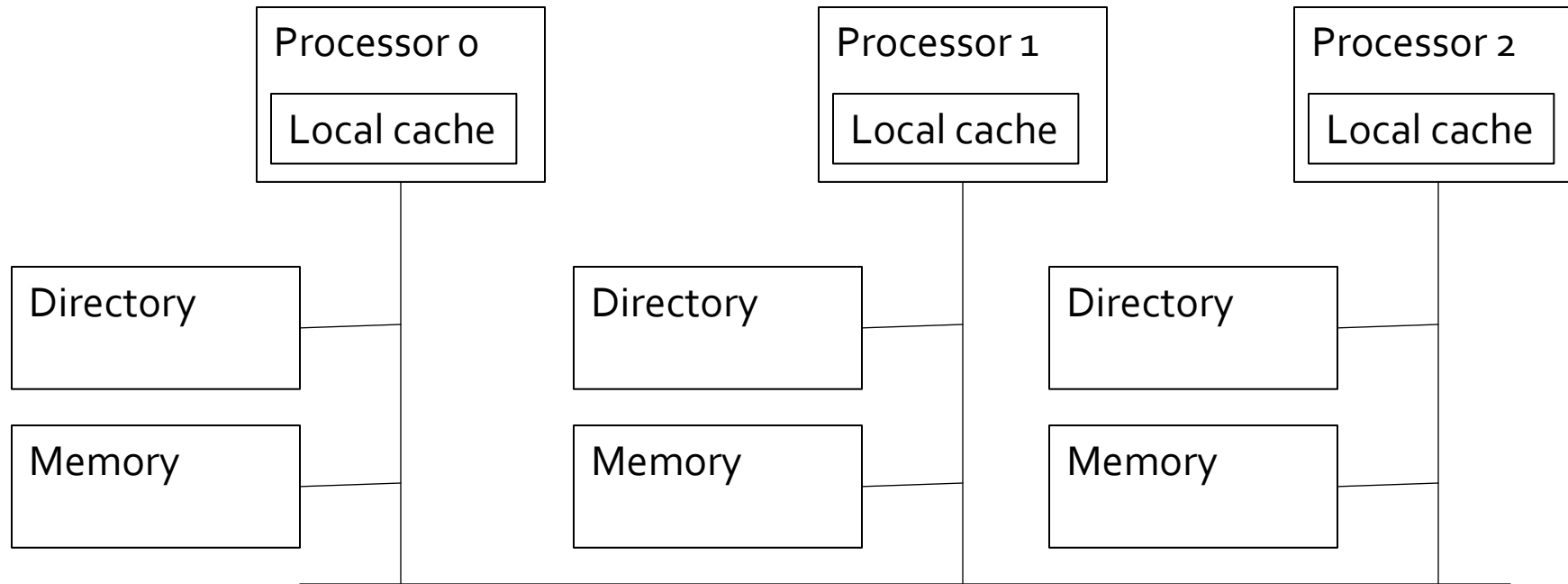
- Avoid broadcast by storing the information about the status of a line in cache in a directory
- A directory entry for a cache line contains info about the state of the cache line in all caches
- Caches look for info in the directories
- Cache coherence is maintained by point-to-point messages between caches

Simple directory

- Directory:
 - Presence bit (one for each processor)
 - Dirty bit (indicates line dirty in one of the processors' caches)
 - One directory entry for cache line of memory
- Cache line states (MSI or MESI)

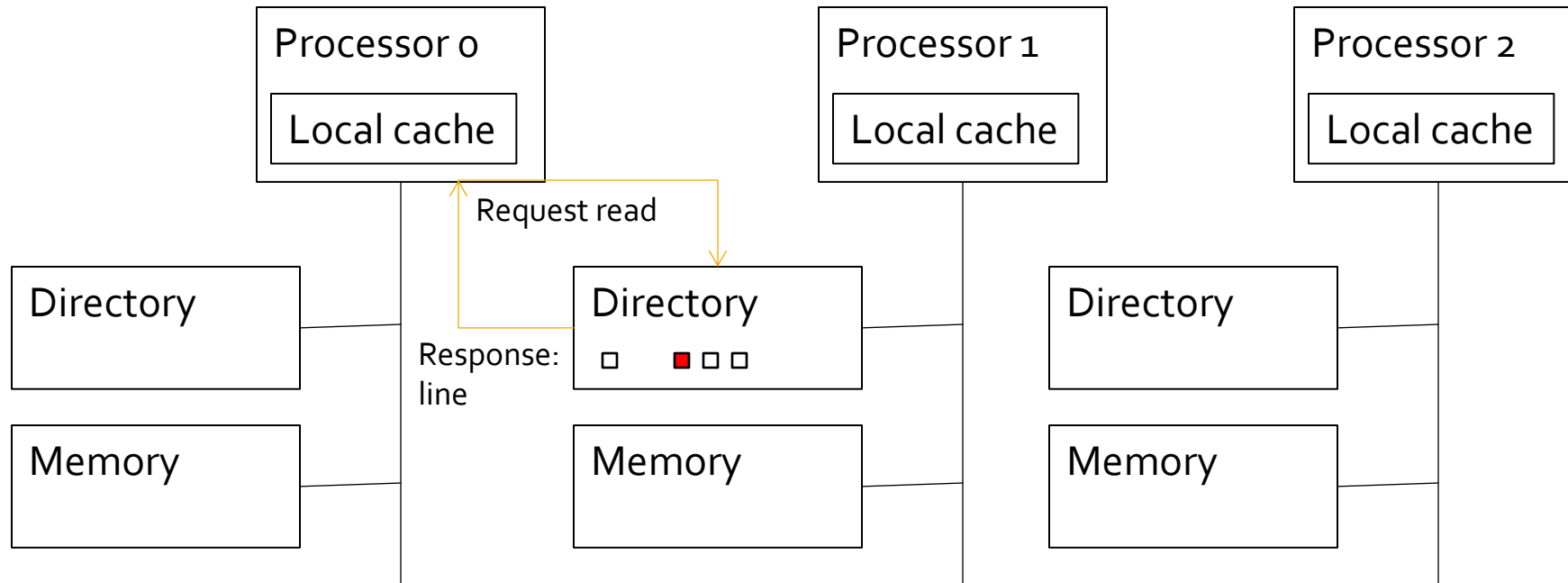


Distributed directory

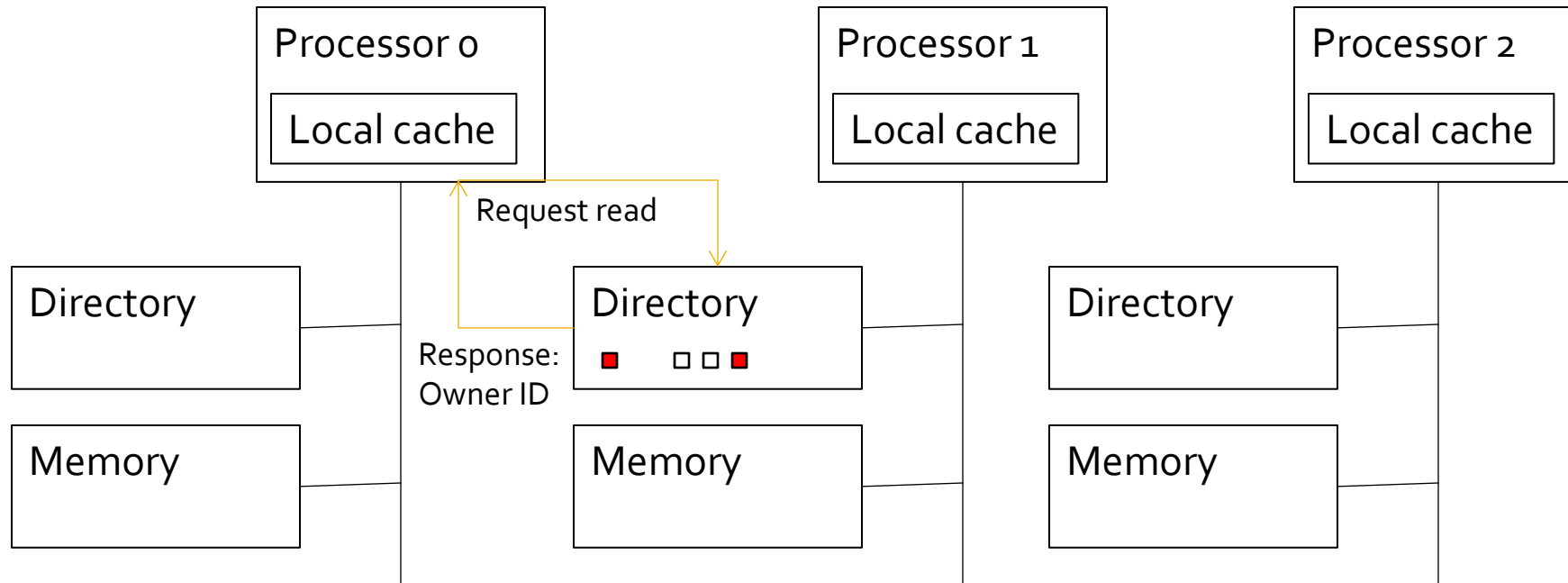


- Home node – node containing the memory holding data (cache line)
- Request node – node containing processor requesting cache line

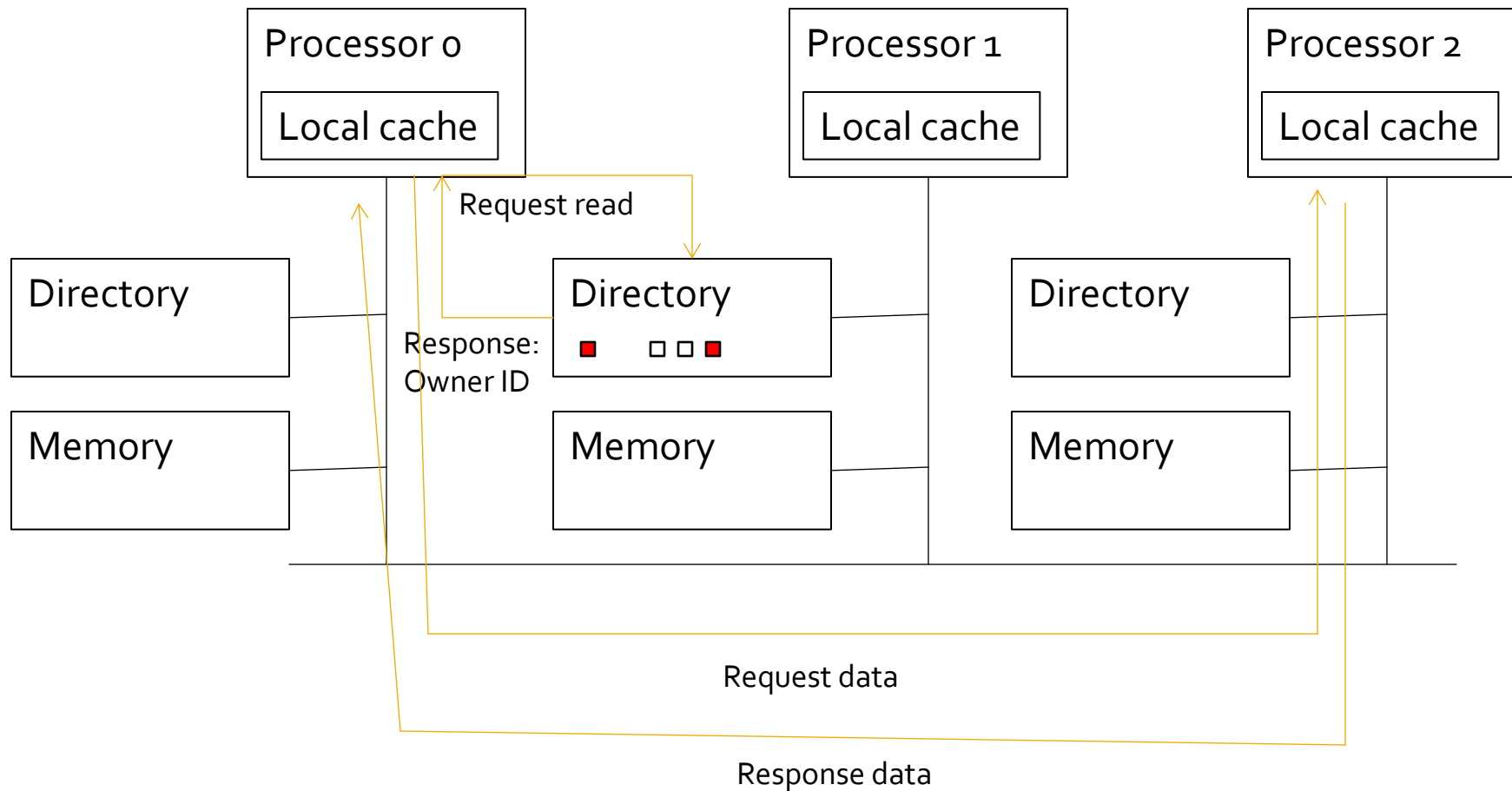
Example: read miss to clean line



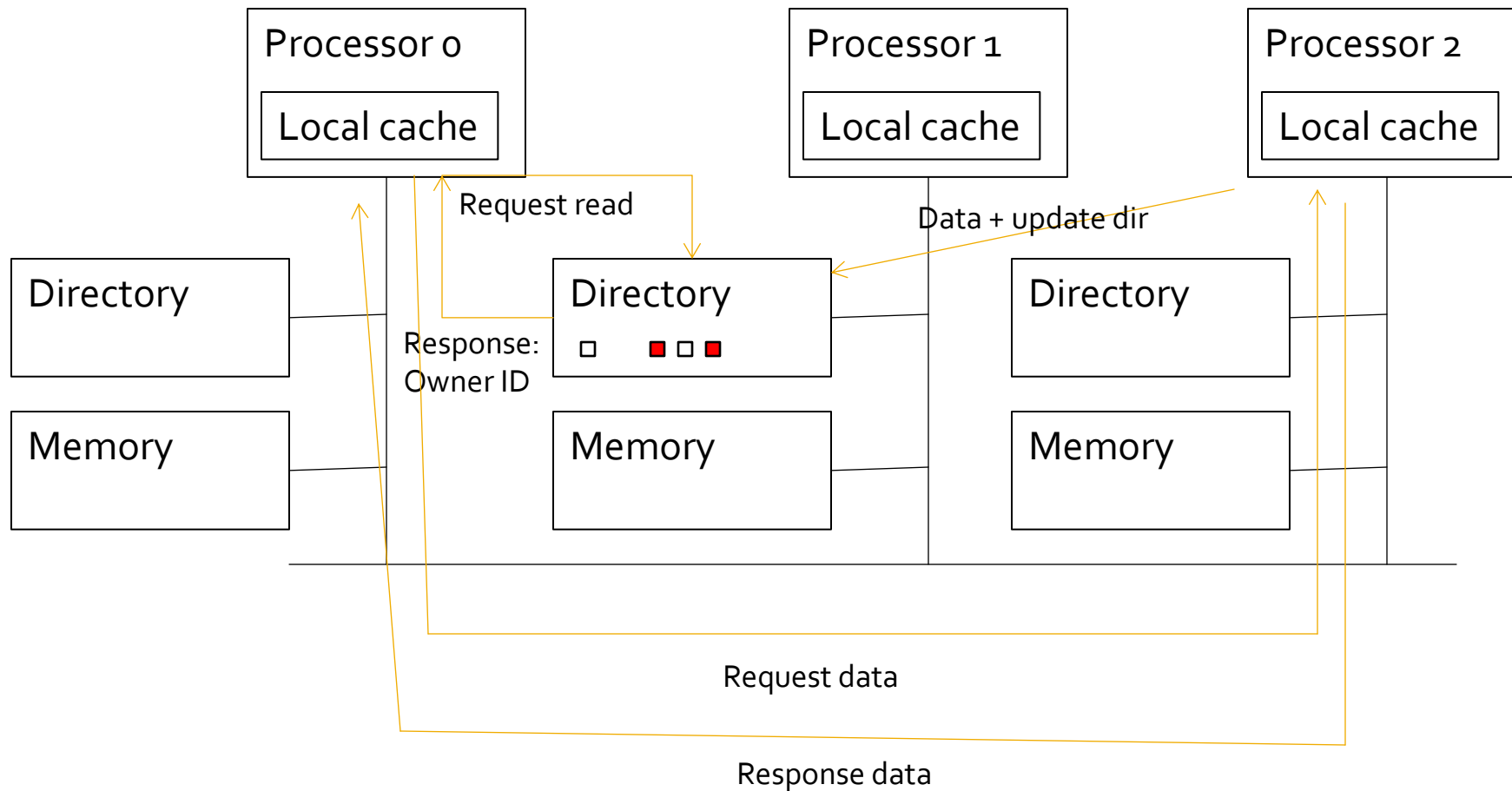
Example: read miss to dirty line



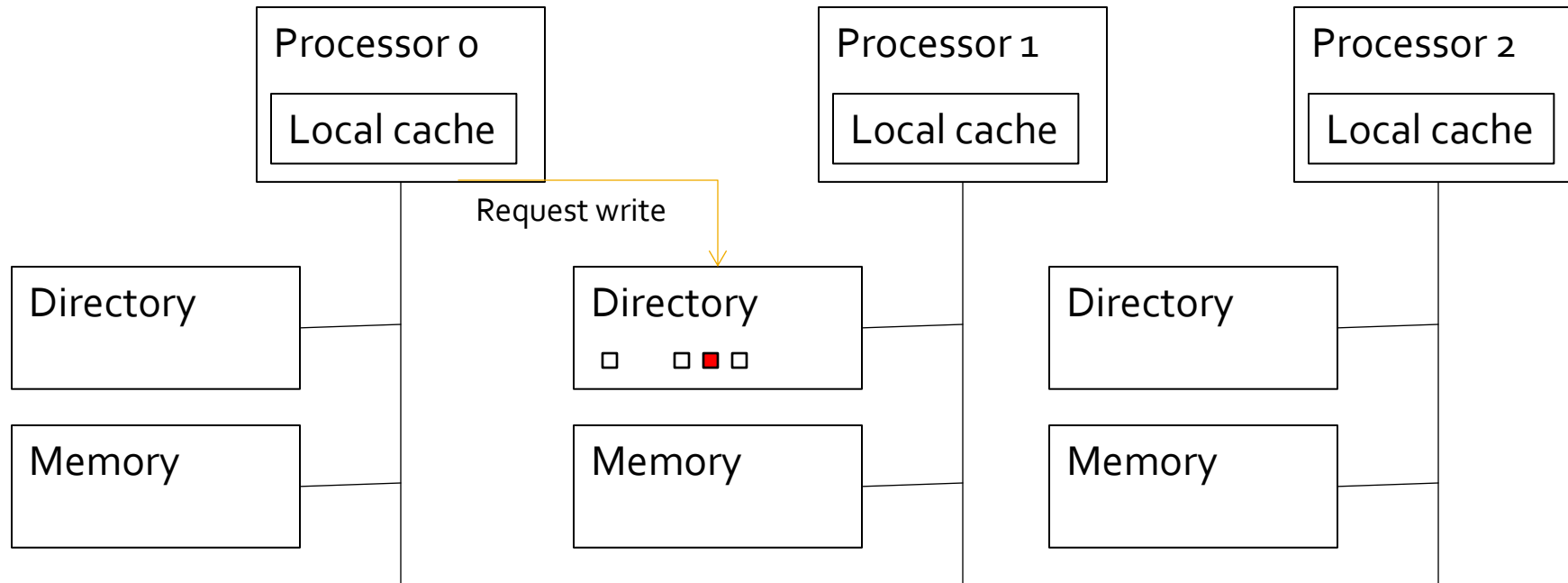
Example: read miss to dirty line



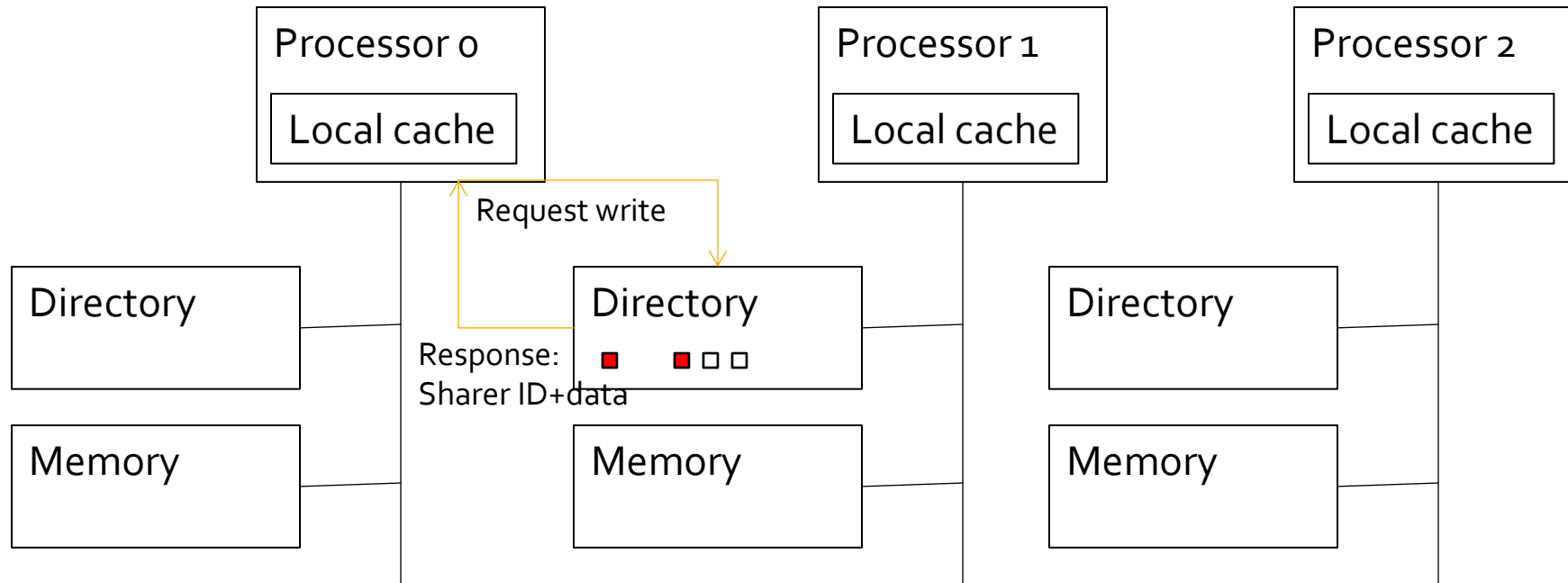
Example: read miss to dirty line



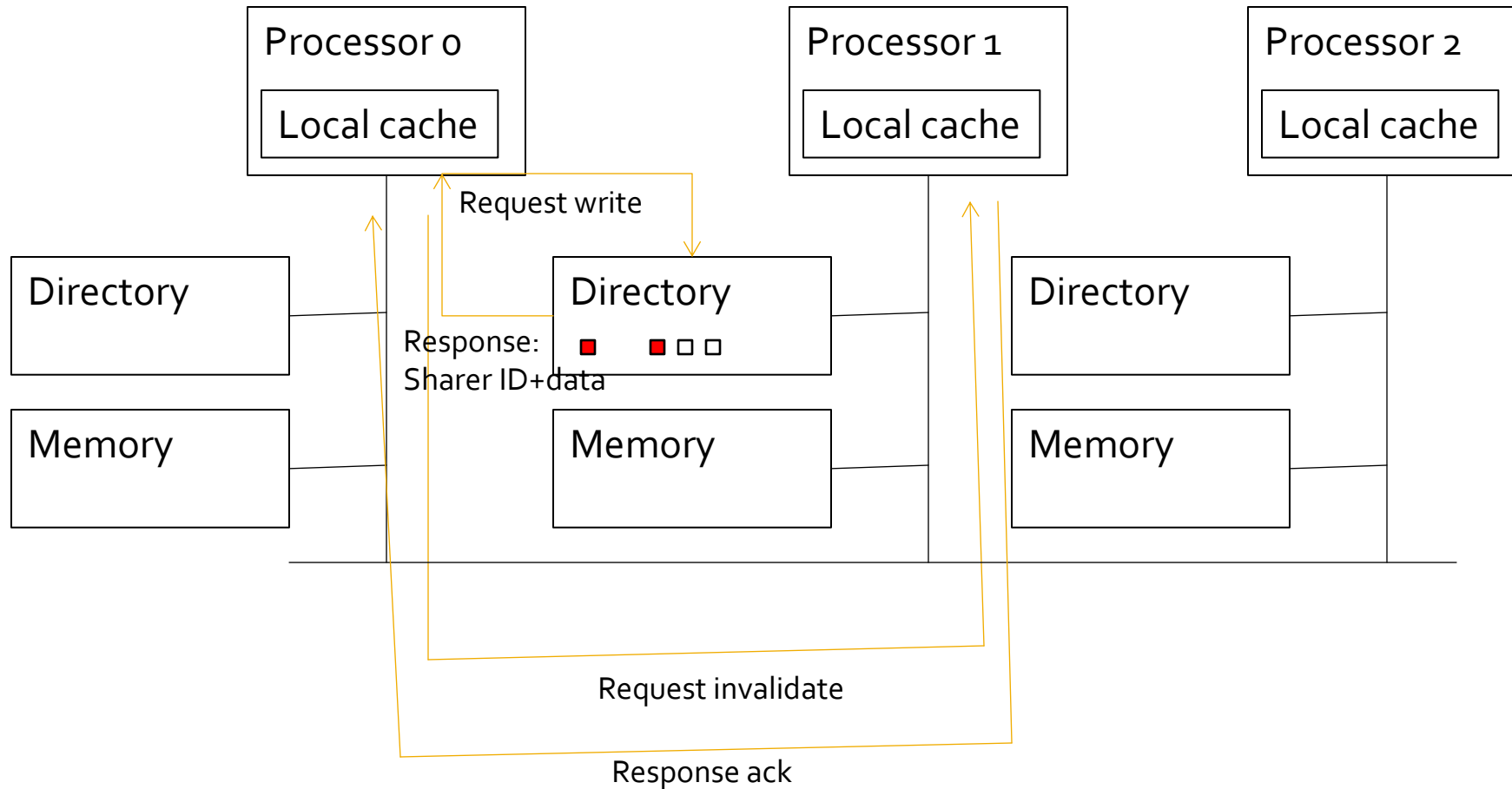
Example: write miss



Example: write miss



Example: write miss



Advantage of directories

- Reads:
 - Directory tells requesting node where to get line from
 - Point-to-point communication
- Writes:
 - Depends on the number of shares
 - If all caches share data, just like snooping protocol